Banking Analytics Dashboard

Project Overview

This project focuses on building a banking analytics dashboard using SQL Server and Power BI. The main objective of the project was to analyze customer behavior, transaction patterns, account activity, and financial trends through interactive dashboards and KPI driven insights.

The project workflow included synthetic data generation, SQL based data cleaning, data integration using joins, Power Query transformations, KPI creation using DAX, dashboard development in Power BI, etc.

The final dashboard was designed to provide insights related to customer demographics, transaction activity, account distribution, inactive accounts, balance analysis, etc.

The project also included intentionally inconsistent and incomplete data to simulate real world banking datasets and create a more practical analytics workflow.

Business Problem

Banks generate large amounts of customer and transaction data on a daily basis. However, raw data alone is not useful unless it can be cleaned, organized, and transformed into meaningful business insights.

The objective of this project was to build a banking analytics dashboard capable of analyzing customer transaction behavior, account performance, customer demographics, inactive accounts, financial trends, etc.

The project also focused on handling real-world data quality issues such as inconsistent date formats, missing values, invalid relationships, duplicate-like records, inconsistent text formatting, etc.

By transforming raw banking data into structured visual insights, the dashboard helps simulate how business intelligence systems are used in financial organizations for monitoring operations and customer activity.

Project Objectives

The main objectives of this project were:

- Generate a realistic banking dataset using SQL
- Simulate real world data inconsistencies and quality issues
- Clean and standardize the data using SQL and Power Query
- Combine multiple banking tables using SQL joins
- Connect SQL Server with Power BI for analysis
- Create interactive KPIs and visualizations using DAX
- Analyze customer demographics and transaction behavior
- Identify inactive accounts and unusual balance patterns
- Build a structured and insight driven banking dashboard
- Improve understanding of end to end data analytics workflows

---

Data Generation & Dataset Design

To make the project more realistic, I decided not to use a perfectly clean dataset. Instead, I generated synthetic banking data using SQL along with AI assisted prompt engineering through Perplexity.

The goal was to simulate a banking environment where datasets often contain inconsistencies, missing values, formatting issues, and relationship mismatches.

The dataset was divided into three main tables:

- Customers

```sql
CREATE TABLE Customers (
    CustomerID      INT PRIMARY KEY,
    Name            NVARCHAR(100),
    Gender          VARCHAR(10) NULL,
    DateOfBirth     VARCHAR(20),
    Address         NVARCHAR(200) NULL,
    Email           NVARCHAR(100) NULL,
    Phone           VARCHAR(20),
    AccountID       INT
);
```

```
INSERT INTO Customers (CustomerID, Name, Gender, DateOfBirt
h, Address, Email, Phone, AccountID) VALUES
(1, 'Ajay Sharma', 'M', '1980-11-04', '123 Main St', NULL,
'9891000001', 101),
(2, 'priya singh', NULL, '21-07-1975', '22B Park Road', 'pr
iya@sample.com', '9891000002', 102),
(3, 'Sarah Khan', 'F', '1989/02/20', '', 'sarahkhan@email.c
om', '9891000003', 103),
(4, 'Mark Lee', 'M', '1995-05-14', '456 North Rd', 'markl@e
mail.com', '9891000004', 104),
(5, 'NAdea KUmar', 'F', '04-12-1982', NULL, NULL, '98910000
05', 105);
```

- Accounts

```
CREATE TABLE Accounts (
    AccountID   INT PRIMARY KEY,
    CustomerID  INT,
    Type        NVARCHAR(20),
    OpenDate    VARCHAR(20),
    Balance     DECIMAL(18,2)
);

INSERT INTO Accounts (AccountID, CustomerID, Type, OpenDat
e, Balance) VALUES
(101, 1, 'SAVINGS',    '03/14/2013',      10000.00),
(102, 2, 'current',    '2011-06-20',    -157874.40),
(103, 3, 'Savings',    '2019/11/02',      0.00),
(104, 4, 'CURRENT',    '21-07-2018',        50.00),
(105, 99, 'Savings',   '07-05-2016',       14.75);
```

- Transactions

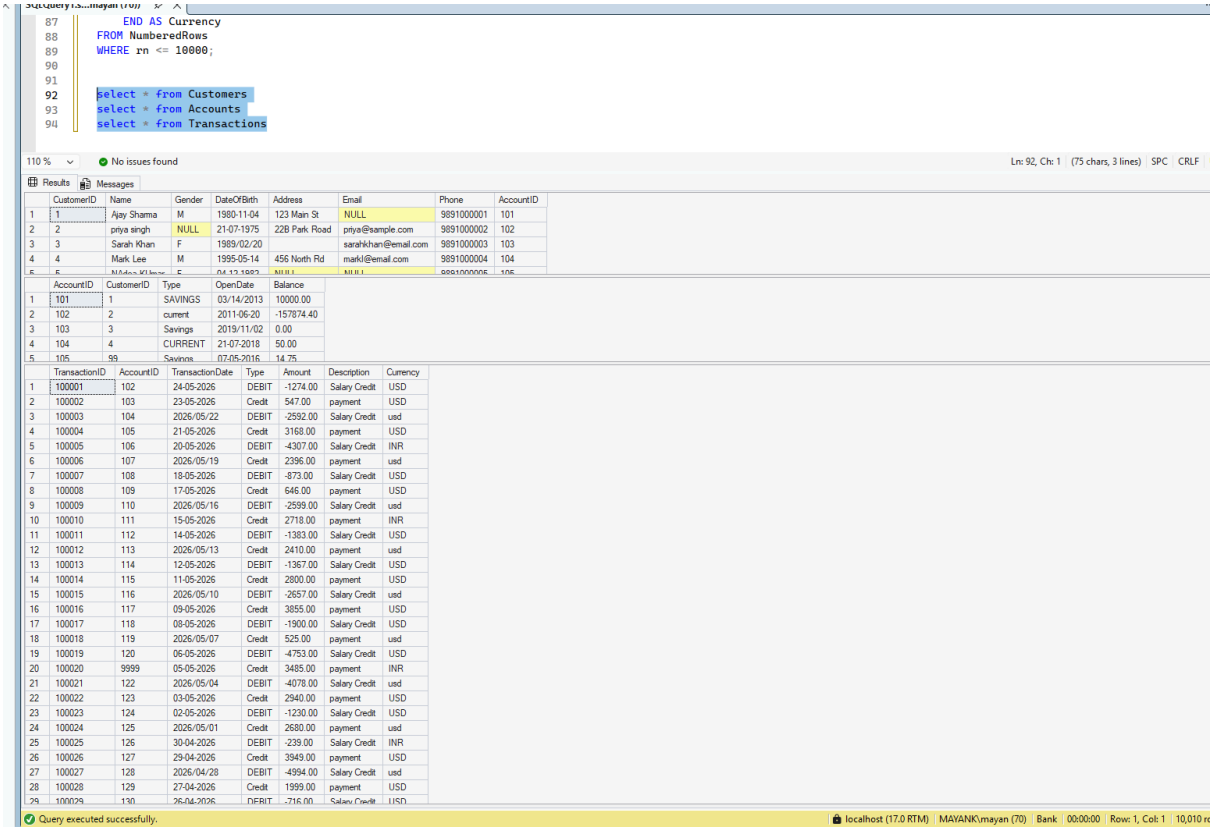
```
CREATE TABLE Transactions (
    TransactionID    INT,
    AccountID        INT,
```

```
    TransactionDate  VARCHAR(20),
    Type             VARCHAR(20),
    Amount           DECIMAL(18,2),
    Description      NVARCHAR(200) NULL,
    Currency         VARCHAR(10),
);
;WITH NumberedRows AS (
    SELECT
        ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS rn
    FROM
        sys.all_objects a
        CROSS JOIN sys.all_columns c
)
INSERT INTO Transactions (
    TransactionID, AccountID, TransactionDate, Type, Amoun
t, Description, Currency
)
SELECT
    100000 + rn AS TransactionID,
    CASE WHEN rn % 20 = 0 THEN 9999 ELSE 101 + (rn % 100) E
ND AS AccountID,
    CASE
        WHEN rn % 3 = 0 THEN FORMAT(GETDATE() - (rn % 365),
'yyyy/MM/dd')
        ELSE CONVERT(VARCHAR(10), GETDATE() - (rn % 365), 1
05)
    END AS TransactionDate,
    CASE WHEN rn % 2 = 0 THEN 'Credit' ELSE 'DEBIT' END AS
Type,
    CASE
        WHEN rn % 1000 = 0 THEN -99999.99
        WHEN rn % 250 = 0 THEN 1000000.99
        ELSE (ABS(CHECKSUM(NEWID())) % 5000) *
             CASE WHEN rn % 2 = 0 THEN 1 ELSE -1 END
    END AS Amount,
    CASE
        WHEN rn % 50 = 0 THEN NULL
        ELSE CASE WHEN rn % 2 = 0 THEN 'payment' ELSE 'Sala
```

```
ry Credit' END
    END AS Descriptio
    CASE
        WHEN rn % 3 = 0 THEN 'usd'
        WHEN rn % 5 = 0 THEN 'INR'
        ELSE 'USD'
    END AS Currency
FROM NumberedRows
WHERE rn <= 10000;
```

Each table was designed to represent a different part of the banking system and later connected during the analysis phase using SQL joins and Power BI relationships.

Instead of generating ideal data, several intentional data quality issues were introduced to create a more practical analytics workflow. Some of these included Mixed date formats, Missing values Duplicate transaction possibilities, Inconsistent text capitalization, Invalid foreign key relationships, Negative and outlier transaction amounts, Null descriptions, Different currency formats etc.



Data Quality Assessment

Standardizing Date Formats-

One of the first issues I noticed in the dataset was the inconsistency in date formats across multiple tables.

All the tables contained dates stored in different formats such as:

- yyyy-mm-dd
- yyyy/mm/dd
- dd-mm-yyyy
- mm/dd/yyyy

Since Power BI performs better with standardized date formats, these inconsistencies needed to be cleaned before importing the data into the reporting layer.

To handle this, I used SQL's TRY_CONVERT() function to safely identify and convert multiple date patterns into a single consistent format (MM/DD/YYYY).

Instead of directly converting values, I used conditional logic with CASE statements to check different date styles individually. This approach helped avoid query failures caused by invalid or mixed date values.

The cleaning process was applied to:

- OPENDATE in the Accounts table
- DATEOFBIRTH in the Customers table
- TRANSACTIONDATE in the Transactions table

```
UPDATE Accounts
SET OpenDate =
    CASE
        WHEN TRY_CONVERT(date, OpenDate, 101) IS NOT NULL
            THEN FORMAT(TRY_CONVERT(date, OpenDate, 101),
'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, OpenDate, 23) IS NOT NULL
            THEN FORMAT(TRY_CONVERT(date, OpenDate, 23), 'M
M/dd/yyyy')
        WHEN TRY_CONVERT(date, OpenDate, 111) IS NOT NULL
```

```
            THEN FORMAT(TRY_CONVERT(date, OpenDate, 111),
'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, OpenDate, 105) IS NOT NULL
            THEN FORMAT(TRY_CONVERT(date, OpenDate, 105),
'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, OpenDate, 103) IS NOT NULL
            THEN FORMAT(TRY_CONVERT(date, OpenDate, 103),
'MM/dd/yyyy')
        ELSE OpenDate
    END;

UPDATE Customers
SET DateOfBirth =
    CASE
        WHEN TRY_CONVERT(date, DateOfBirth, 101) IS NOT NUL
L
            THEN FORMAT(TRY_CONVERT(date, DateOfBirth, 10
1), 'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, DateOfBirth, 23) IS NOT NULL
            THEN FORMAT(TRY_CONVERT(date, DateOfBirth, 23),
'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, DateOfBirth, 111) IS NOT NUL
L
            THEN FORMAT(TRY_CONVERT(date, DateOfBirth, 11
1), 'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, DateOfBirth, 105) IS NOT NUL
L
            THEN FORMAT(TRY_CONVERT(date, DateOfBirth, 10
5), 'MM/dd/yyyy')
        WHEN TRY_CONVERT(date, DateOfBirth, 103) IS NOT NUL
L
            THEN FORMAT(TRY_CONVERT(date, DateOfBirth, 10
3), 'MM/dd/yyyy')
        ELSE DateOfBirth
    END;

UPDATE Transactions
SET Transactiondate =
```

```
    CASE
        WHEN TRY_CONVERT(date, Transactiondate, 101) IS NOT
NULL
            THEN FORMAT(TRY_CONVERT(date, Transactiondate,
101), 'MM/dd/yyyy')

        WHEN TRY_CONVERT(date, Transactiondate, 23) IS NOT
NULL
            THEN FORMAT(TRY_CONVERT(date, Transactiondate,
23), 'MM/dd/yyyy')

        WHEN TRY_CONVERT(date, Transactiondate, 111) IS NOT
NULL
            THEN FORMAT(TRY_CONVERT(date, Transactiondate,
111), 'MM/dd/yyyy')

        WHEN TRY_CONVERT(date, Transactiondate, 105) IS NOT
NULL
            THEN FORMAT(TRY_CONVERT(date, Transactiondate,
105), 'MM/dd/yyyy')

        WHEN TRY_CONVERT(date, Transactiondate, 103) IS NOT
NULL
            THEN FORMAT(TRY_CONVERT(date, Transactiondate,
103), 'MM/dd/yyyy')
        ELSE Transactiondate
    END;
```

Result after code execution

```
64
65  SELECT DISTINCT OpenDate FROM Accounts
66  SELECT DISTINCT DateOfBirth FROM Customers
67  SELECT DISTINCT TransactionDate FROM Transactions
```

110 % No issues found

Results Messages

	OpenDate
1	03/14/2013
2	06/20/2011
3	07/05/2016
4	07/21/2018
5	11/02/2019

	DateOfBirth
1	02/20/1989
2	04/12/1982
3	05/14/1995
4	07/21/1975
5	11/04/1980

	TransactionDate
1	09/21/2025
2	12/03/2026
3	01/10/2026
4	11/10/2025
5	09/15/2025
6	07/14/2025
7	03/06/2025
8	12/04/2026
9	01/07/2025
10	08/06/2025
11	10/23/2025
12	10/08/2025
13	09/19/2025
14	07/03/2026
15	11/28/2025
16	07/23/2025
17	04/08/2025
18	03/30/2026
19	07/18/2025
20	01/04/2026
21	12/25/2025
22	08/24/2025
23	09/01/2025
24	02/01/2026
25	09/02/2025
26	04/24/2026
27	11/02/2026
28	01/20/2026
29	10/21/2025
30	08/03/2025
31	08/13/2025

Combining Banking Data Using SQL Joins

After cleaning the individual tables, the next step was to combine the banking data into a single analytical dataset.

Since each table represented a different part of the banking system, they needed to be connected before performing analysis in Power BI.

To achieve this, I used SQL joins to merge the datasets based on their related account and customer information.

The Transactions table was connected to the Accounts table using `AccountID`, and the Accounts table was then connected to the Customers table using `CustomerID`.

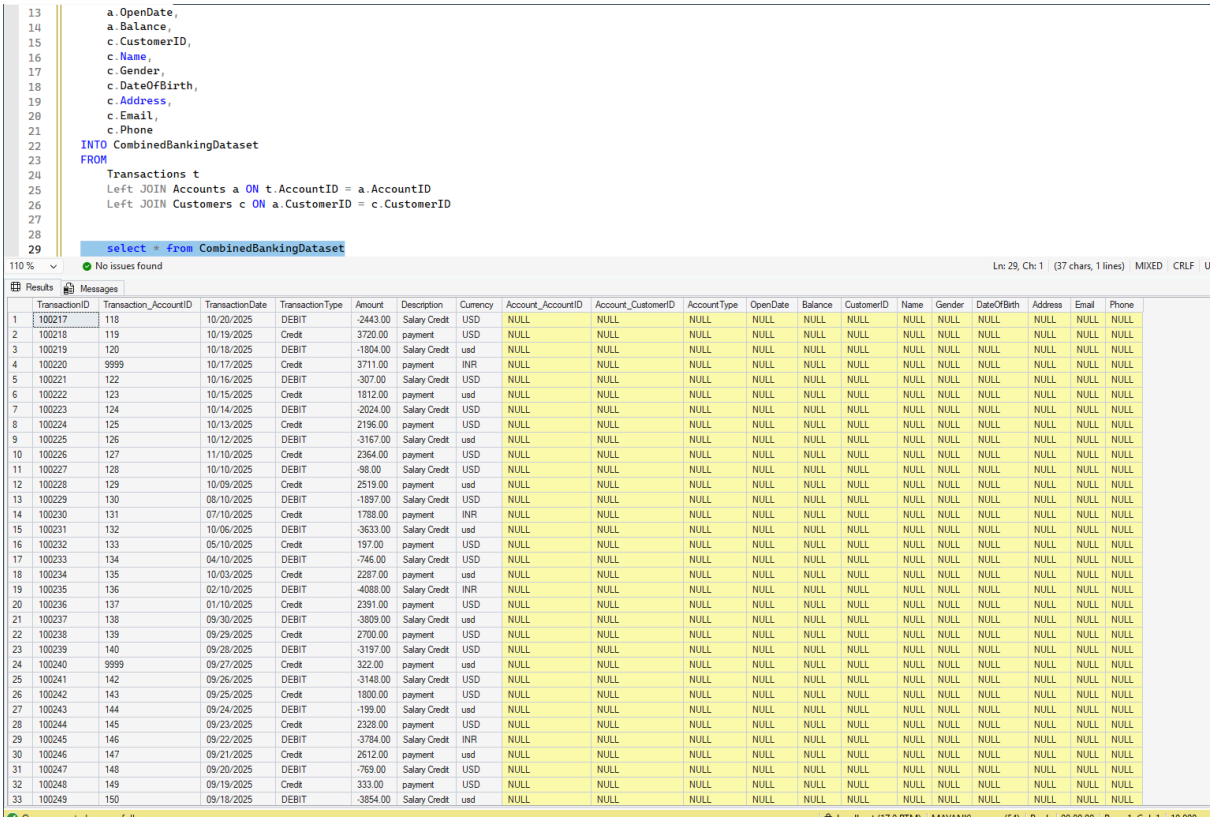
I specifically used `LEFT JOIN` operations instead of `INNER JOIN` because the dataset intentionally contained mismatched and missing relationships. Using left joins allowed all transaction records to remain in the final dataset even if corresponding account or customer records were missing.

The final merged table was stored as `CombinedBankingDataset.`

```
SELECT
    t.TransactionID,
    t.AccountID AS Transaction_AccountID,
    t.TransactionDate,
    t.Type AS TransactionType,
    t.Amount,
    t.Description,
    t.Currency,
    a.AccountID AS Account_AccountID,
    a.CustomerID AS Account_CustomerID,
    a.Type AS AccountType,
    a.OpenDate,
    a.Balance,
    c.CustomerID,
    c.Name,
    c.Gender,
    c.DateOfBirth,
    c.Address,
    c.Email,
    c.Phone
INTO CombinedBankingDataset
FROM
    Transactions t
    Left JOIN Accounts a ON t.AccountID = a.AccountID
```

```
    Left JOIN Customers c ON a.CustomerID = c.CustomerID

    select * from CombinedBankingDataset
```



```
    a.OpenDate,
    a.Balance,
    c.CustomerID,
    c.Name,
    c.Gender,
    c.DateOfBirth,
    c.Address,
    c.Email,
    c.Phone
INTO CombinedBankingDataset
FROM
    Transactions t
    Left JOIN Accounts a ON t.AccountID = a.AccountID
    Left JOIN Customers c ON a.CustomerID = c.CustomerID


    select * from CombinedBankingDataset
```

110 % No issues found Ln: 29, Ch: 1 (37 chars, 1 lines) MIXED CRLF

Results Messages

	TransactionID	Transaction_AccountID	TransactionDate	TransactionType	Amount	Description	Currency	Account_AccountID	Account_CustomerID	AccountType	OpenDate	Balance	CustomerID	Name	Gender	DateOfBirth	Address	Email	Phone
1	100217	118	10/20/2025	DEBIT	-2443.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
2	100218	119	10/19/2025	Credit	3720.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
3	100219	120	10/18/2025	DEBIT	-1804.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
4	100220	9999	10/17/2025	Credit	3711.00	payment	INR	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
5	100221	122	10/16/2025	DEBIT	-307.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
6	100222	123	10/15/2025	Credit	1812.00	payment	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
7	100223	124	10/14/2025	DEBIT	-2024.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
8	100224	125	10/13/2025	Credit	2196.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
9	100225	126	10/12/2025	DEBIT	-3167.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
10	100226	127	11/10/2025	Credit	2364.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
11	100227	128	10/10/2025	DEBIT	-98.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12	100228	129	10/09/2025	Credit	2519.00	payment	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
13	100229	130	08/10/2025	DEBIT	-1897.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
14	100230	131	07/10/2025	Credit	1788.00	payment	INR	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
15	100231	132	10/06/2025	DEBIT	-3633.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
16	100232	133	05/10/2025	Credit	197.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
17	100233	134	04/10/2025	DEBIT	-746.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
18	100234	135	10/03/2025	Credit	2287.00	payment	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
19	100235	136	02/10/2025	DEBIT	-4088.00	Salary Credit	INR	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
20	100236	137	01/10/2025	Credit	2391.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
21	100237	138	09/30/2025	DEBIT	-3809.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
22	100238	139	09/29/2025	Credit	2700.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
23	100239	140	09/28/2025	DEBIT	-3197.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
24	100240	9999	09/27/2025	Credit	322.00	payment	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
25	100241	142	09/26/2025	DEBIT	-3148.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
26	100242	143	09/25/2025	Credit	1800.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
27	100243	144	09/24/2025	DEBIT	-199.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
28	100244	145	09/23/2025	Credit	2328.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
29	100245	146	09/22/2025	DEBIT	-3784.00	Salary Credit	INR	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
30	100246	147	09/21/2025	Credit	2612.00	payment	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
31	100247	148	09/20/2025	DEBIT	-769.00	Salary Credit	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
32	100248	149	09/19/2025	Credit	333.00	payment	USD	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
33	100249	150	09/18/2025	DEBIT	-3854.00	Salary Credit	usd	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Connecting SQL Server to Power BI

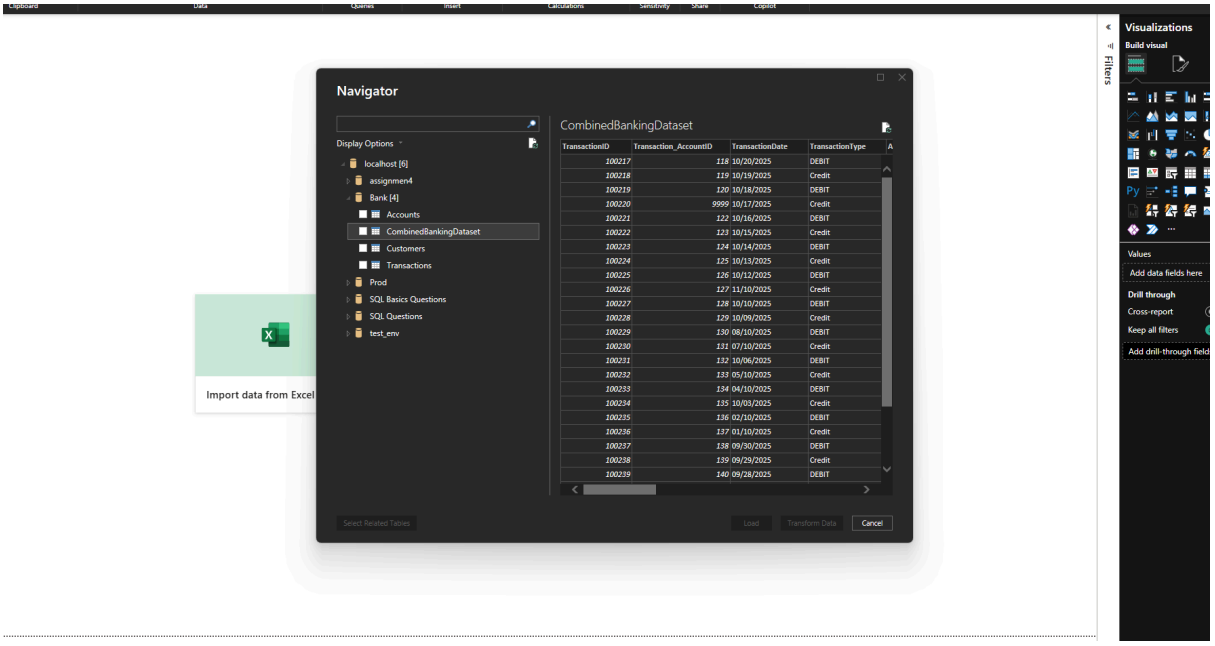
Once the data cleaning and table integration process was completed in SQL Server, the next step was to connect the database to Power BI for visualization and analysis.

Instead of exporting the data manually into Excel or CSV files, I connected Power BI directly to the SQL Server database. This created a more structured and scalable workflow where the reporting layer could directly access the processed data.

The connection was established using the SQL Server connector available inside Power BI Desktop.

After connecting to the database, the `CombinedBankingDataset` table was imported into Power BI for further analysis and dashboard development.

Once the connection was completed successfully, I began building KPIs, visualizations, and business insights using the imported banking dataset.

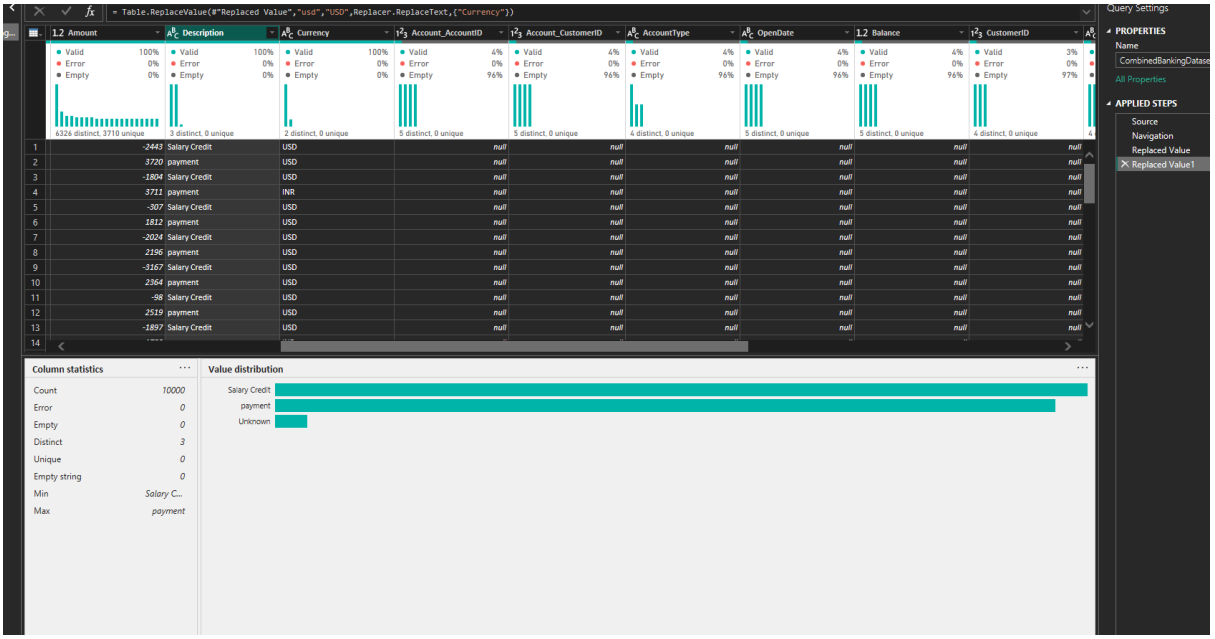


Data Cleaning in Power Query Editor

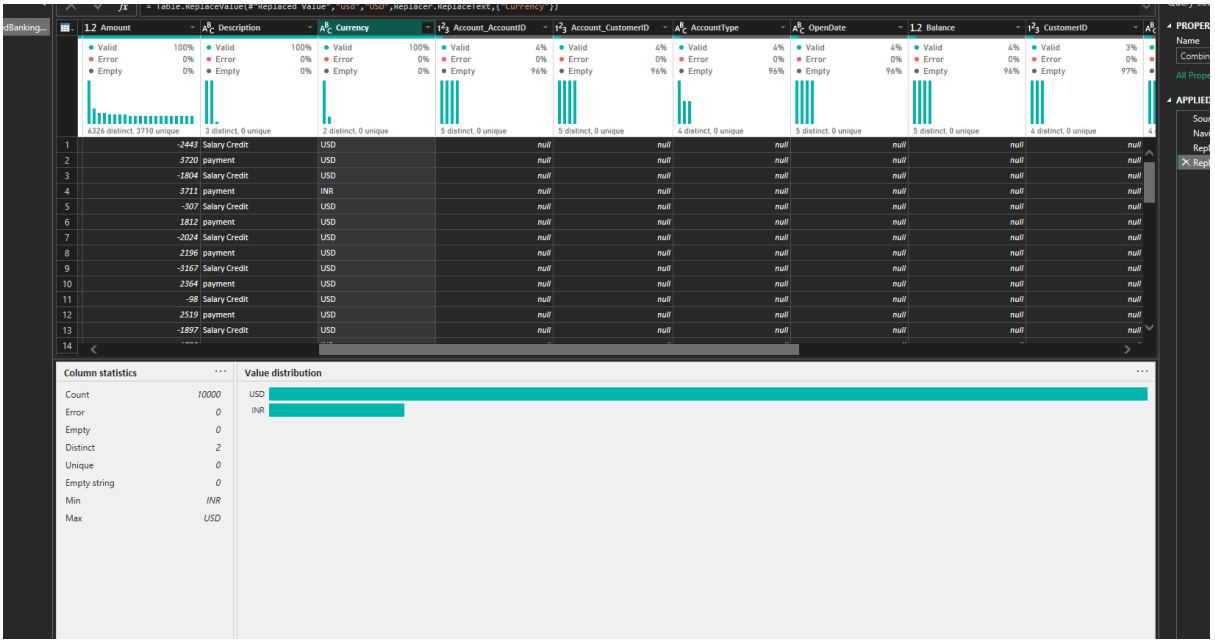
After importing the dataset into Power BI, I used Power Query Editor to perform additional cleaning and transformation before building the dashboard.

Although several inconsistencies had already been handled in SQL Server, some issues became more visible once the data was loaded into Power BI.

One of the first issues I noticed was the presence of null values in the `Description` column. Since blank descriptions could create confusion during analysis and visual filtering, I replaced those null values with the label `"Unknown"` to make the dataset more readable and consistent.



I also found inconsistent currency formatting in the `Currency` column. Some records contained `"USD"` while others were stored as `"usd"`. To standardize the values, all lowercase entries were converted to uppercase so the category would appear correctly in visualizations and aggregations.



Similarly, the `AccountType` column contained inconsistent capitalization such as:

- `current`
- `CURRENT`
- `Current`

These values were standardized to maintain consistency across reports and charts.

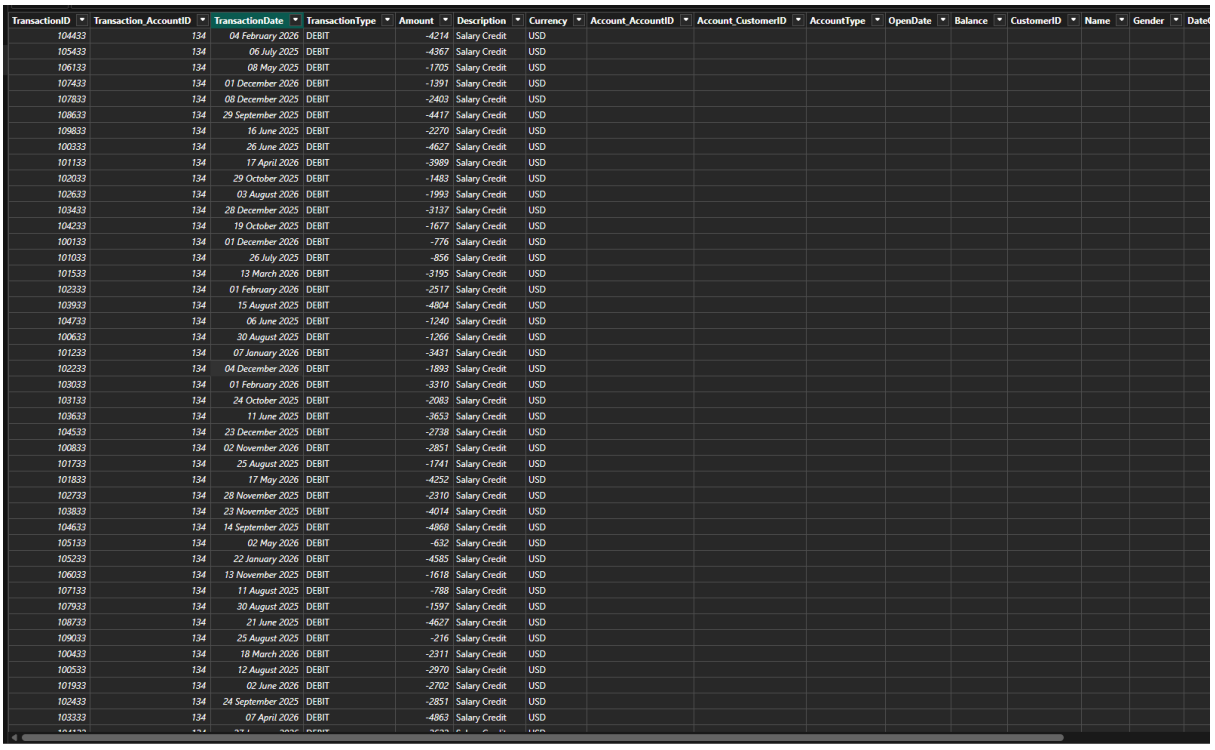
After handling text inconsistencies, I assigned appropriate data types to all columns inside Power Query. This included:

- dates
- numeric values
- text fields
- currency related columns

While converting the `TransactionDate` column, I encountered multiple parsing errors. Even though the dates had already been standardized in SQL, Power BI still interpreted some records incorrectly because of regional date settings.

To solve this issue, I used the "Change Type with Locale" option in Power Query and selected the United States locale (`English - United States`). This allowed Power BI to correctly interpret the `MM/dd/yyyy` date format and removed the conversion errors successfully.

Once the cleaning and datatype validation process was completed, the dataset was ready for dashboard development and KPI creation.



TransactionID	Transaction_AccountID	TransactionDate	TransactionType	Amount	Description	Currency	Account_AccountID	Account_CustomerID	AccountType	OpenDate	Balance	CustomerID	Name	Gender	Date
104433	134	04 February 2026	DEBIT	-4214	Salary Credit	USD									
105433	134	06 July 2025	DEBIT	-4367	Salary Credit	USD									
106133	134	08 May 2025	DEBIT	-1705	Salary Credit	USD									
107433	134	01 December 2026	DEBIT	-1391	Salary Credit	USD									
107833	134	08 December 2025	DEBIT	-2403	Salary Credit	USD									
108633	134	29 September 2025	DEBIT	-4417	Salary Credit	USD									
109833	134	16 June 2025	DEBIT	-2270	Salary Credit	USD									
100333	134	26 June 2025	DEBIT	-4627	Salary Credit	USD									
101133	134	17 April 2026	DEBIT	-3989	Salary Credit	USD									
102033	134	29 October 2025	DEBIT	-1483	Salary Credit	USD									
102633	134	03 August 2026	DEBIT	-1993	Salary Credit	USD									
103433	134	28 December 2025	DEBIT	-3137	Salary Credit	USD									
104233	134	19 October 2025	DEBIT	-1677	Salary Credit	USD									
100133	134	01 December 2026	DEBIT	-776	Salary Credit	USD									
101033	134	26 July 2025	DEBIT	-856	Salary Credit	USD									
101533	134	13 March 2026	DEBIT	-3195	Salary Credit	USD									
102333	134	01 February 2026	DEBIT	-2517	Salary Credit	USD									
103933	134	15 August 2025	DEBIT	-4804	Salary Credit	USD									
104733	134	06 June 2025	DEBIT	-1240	Salary Credit	USD									
100633	134	30 August 2025	DEBIT	-1266	Salary Credit	USD									
101233	134	07 January 2026	DEBIT	-3431	Salary Credit	USD									
102233	134	04 December 2026	DEBIT	-1893	Salary Credit	USD									
103033	134	01 February 2026	DEBIT	-3310	Salary Credit	USD									
103133	134	24 October 2025	DEBIT	-2083	Salary Credit	USD									
103633	134	11 June 2025	DEBIT	-3653	Salary Credit	USD									
104533	134	23 December 2025	DEBIT	-2738	Salary Credit	USD									
100833	134	02 November 2026	DEBIT	-2851	Salary Credit	USD									
101733	134	25 August 2025	DEBIT	-1741	Salary Credit	USD									
101833	134	17 May 2026	DEBIT	-4252	Salary Credit	USD									
102733	134	28 November 2025	DEBIT	-2310	Salary Credit	USD									
103833	134	23 November 2025	DEBIT	-4014	Salary Credit	USD									
104633	134	14 September 2025	DEBIT	-4868	Salary Credit	USD									
105133	134	02 May 2026	DEBIT	-632	Salary Credit	USD									
105233	134	22 January 2026	DEBIT	-4585	Salary Credit	USD									
106033	134	13 November 2025	DEBIT	-1618	Salary Credit	USD									
107133	134	11 August 2025	DEBIT	-788	Salary Credit	USD									
107933	134	30 August 2025	DEBIT	-1597	Salary Credit	USD									
108733	134	21 June 2025	DEBIT	-4627	Salary Credit	USD									
109033	134	25 August 2025	DEBIT	-216	Salary Credit	USD									
100433	134	18 March 2026	DEBIT	-2311	Salary Credit	USD									
100533	134	12 August 2025	DEBIT	-2970	Salary Credit	USD									
101933	134	02 June 2026	DEBIT	-2702	Salary Credit	USD									
102433	134	24 September 2025	DEBIT	-2851	Salary Credit	USD									
103333	134	07 April 2026	DEBIT	-4863	Salary Credit	USD									

KPI Planning & Dashboard Design

Before creating the dashboard visuals, I first identified the key business questions the banking dataset should answer.

The main goal was not just to create charts, but to build KPIs that could help analyze:

- customer activity
- transaction behavior
- account performance
- financial trends

Based on the available data, I selected a set of KPIs that would provide both operational and customer level insights.

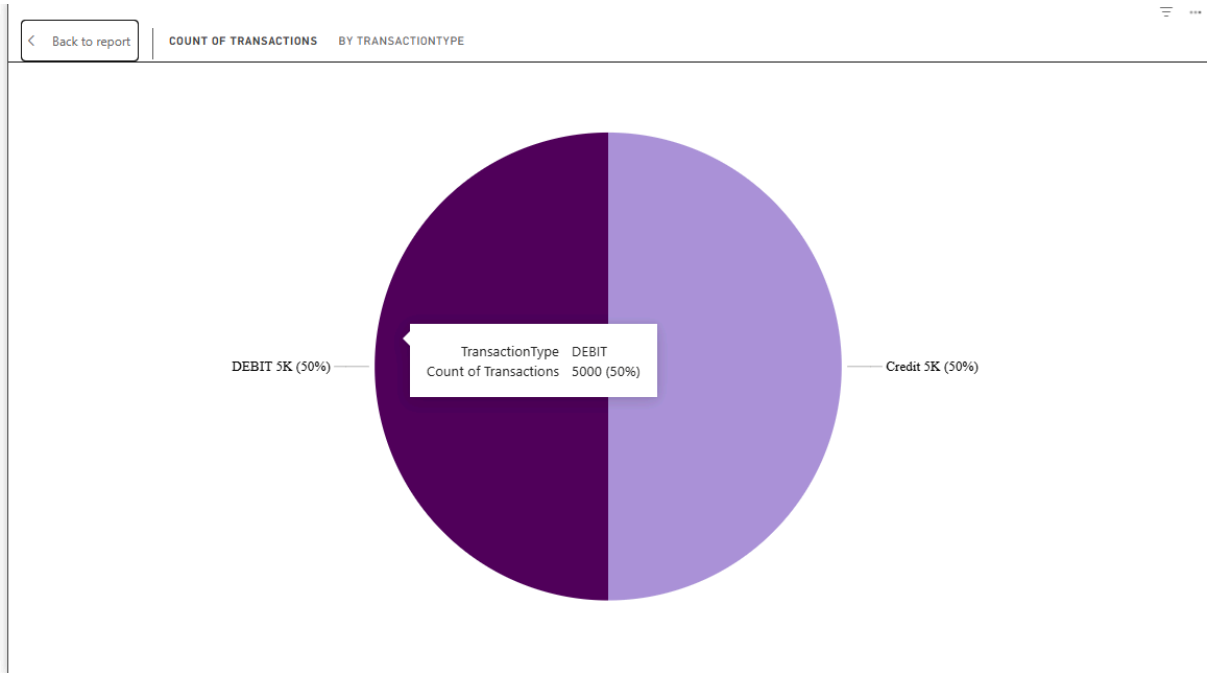
Transactions by Type

This KPI was created to analyze the proportion of Credit and Debit transactions in the system.

A pie chart was used because it provides a quick comparison between transaction categories and helps identify overall transaction patterns.

The metric was calculated using the transaction count grouped by transaction type.

```
Account Count by Type = COUNT(CombinedBankingDataset[Accoun
t_AccountID])
```



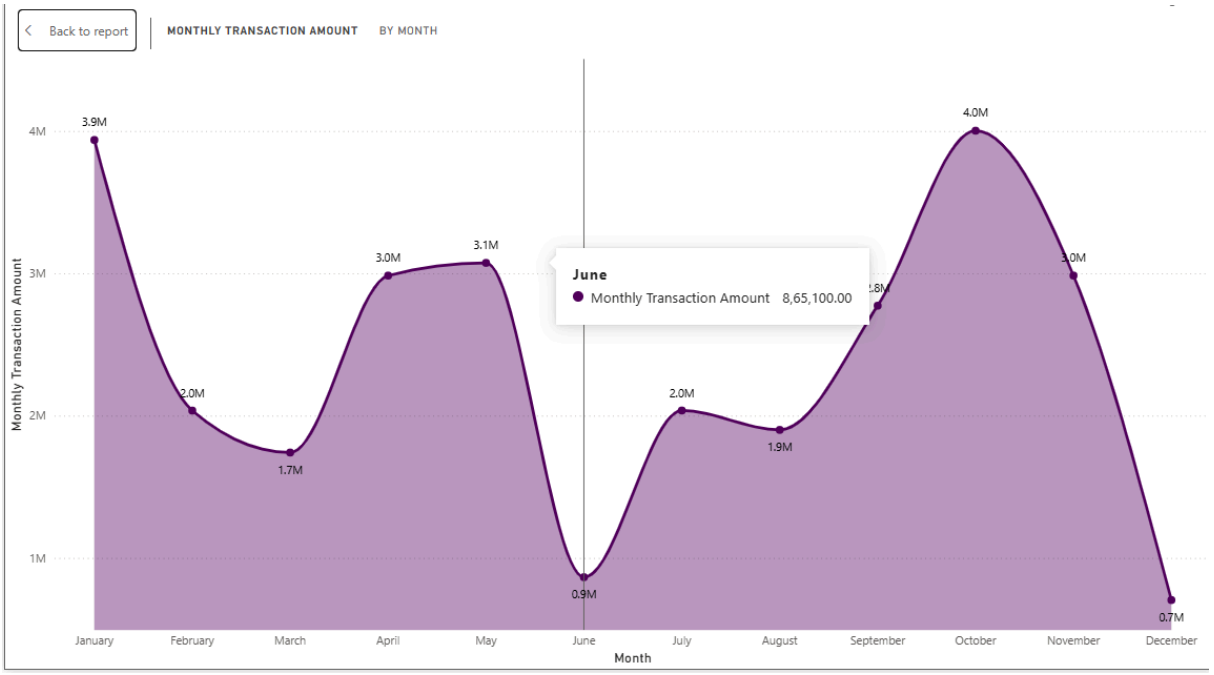
Monthly Transaction Amount

This KPI tracks the total transaction value across different months.

A line chart was selected because it is more suitable for time-based trend analysis and helps visualize increases or decreases in transaction activity over time.

The calculation was built using DAX measures with monthly aggregation logic.

```
Monthly Transaction Amount = CALCULATE(SUM(CombinedBankingD
ataset[Amount]), ALLEXCEPT(CombinedBankingDataset, Combined
BankingDataset[TransactionDate].[Month]))
```

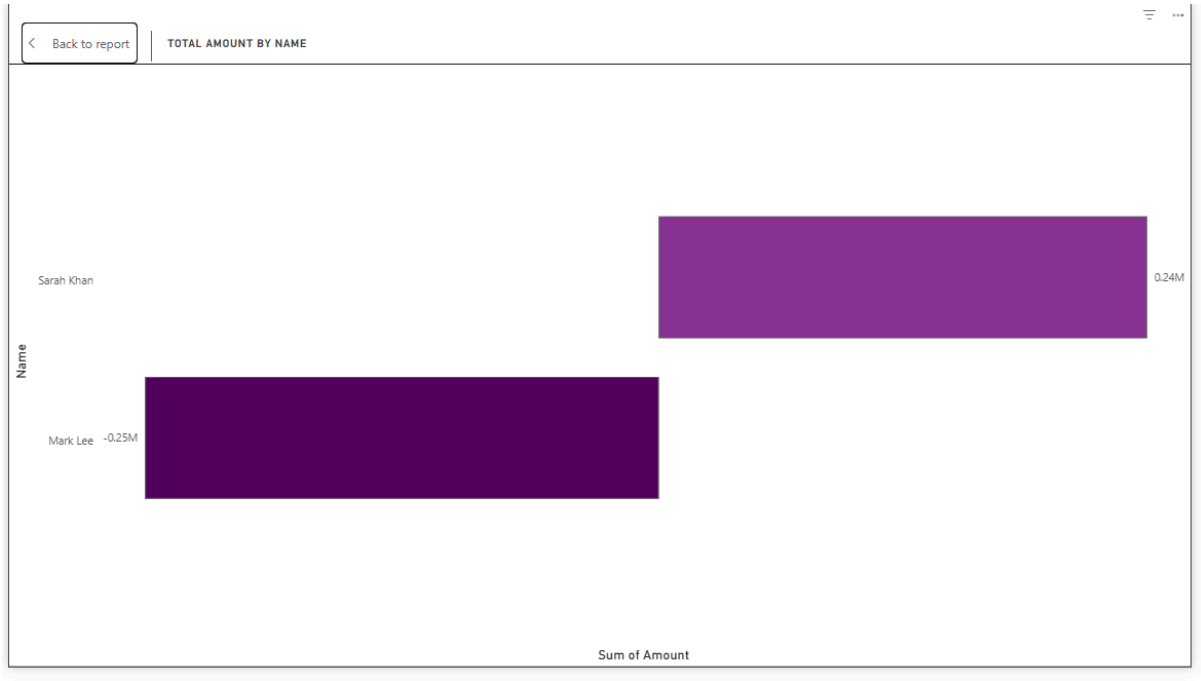


Top Customers by Transaction Value

This KPI identifies customers contributing the highest transaction amounts.

A horizontal bar chart was used to make customer comparisons easier and to highlight the top-performing accounts clearly.

This analysis helps simulate how financial institutions may identify high-value customers.



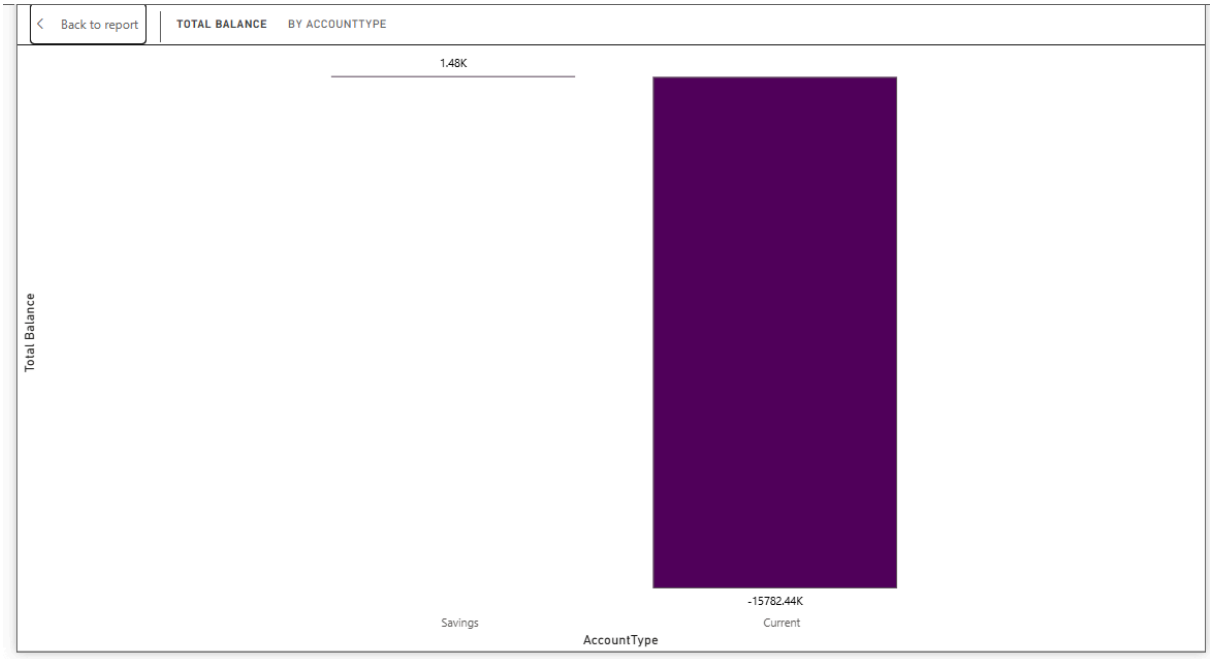
Total Balance by Account Type

This KPI compares the total balances stored in different account categories such as Savings and Current accounts.

A clustered bar chart was chosen to improve category comparison and overall readability.

This visualization helps identify which account types contribute most to the bank's total balance holdings.

```
Total Balance = SUM(CombinedBankingDataset[Balance])
```



Inactive Accounts (Last 90 Days)

This KPI was created to identify accounts with no recent transaction activity within the past 90 days.

The logic was implemented using DAX filters and date comparisons.

This metric is important because inactive accounts can indicate:

- reduced customer engagement
- dormant accounts
- potential churn behavior

A line chart was used to display inactive account information clearly.

```
Inactive Accounts = CALCULATE(DISTINCTCOUNT(CombinedBanking
Dataset[Account_AccountID]), FILTER(VALUES(CombinedBankingD
ataset[Account_AccountID]), CALCULATE(MAX(CombinedBankingDa
taset[TransactionDate])) < TODAY()-90))
```

Customer Gender Distribution

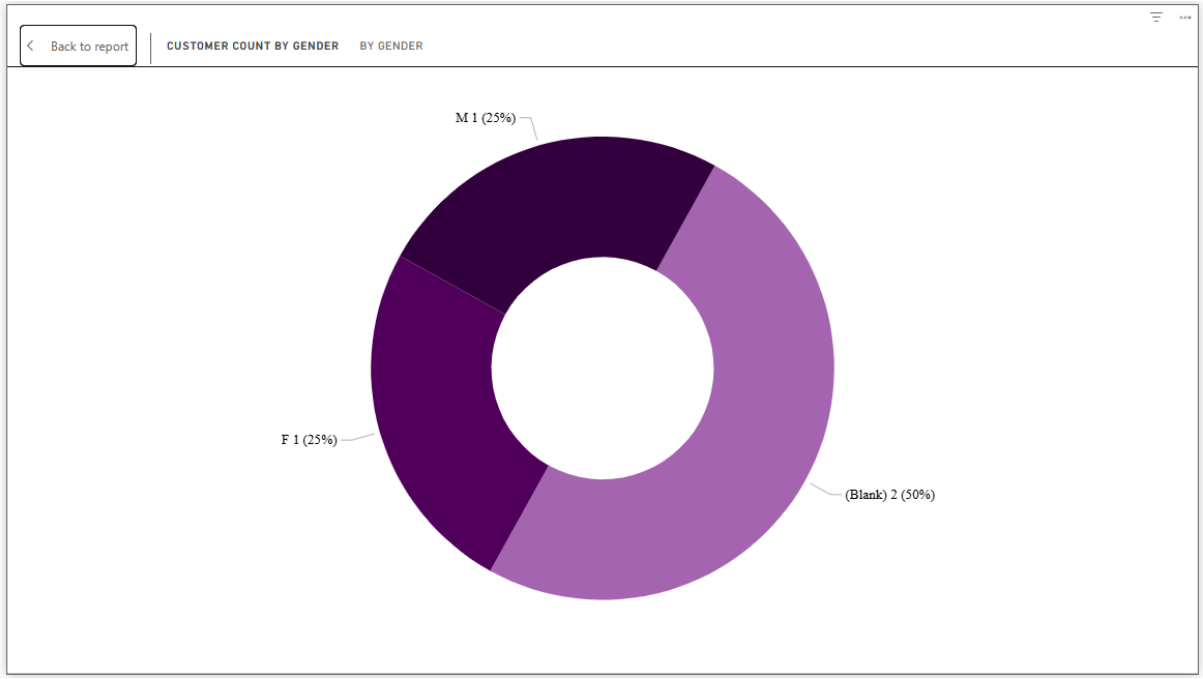
This KPI analyzes the proportion of customers based on gender categories.

A donut chart was used to visualize the distribution clearly and make category comparisons easier.

The metric was calculated using distinct customer counts grouped by gender.

This analysis helps understand customer demographics and overall user composition within the banking dataset.

```
Customer Count by Gender = DISTINCTCOUNT(CombinedBankingDat
aset[CustomerID])
```



Customers by Age Group

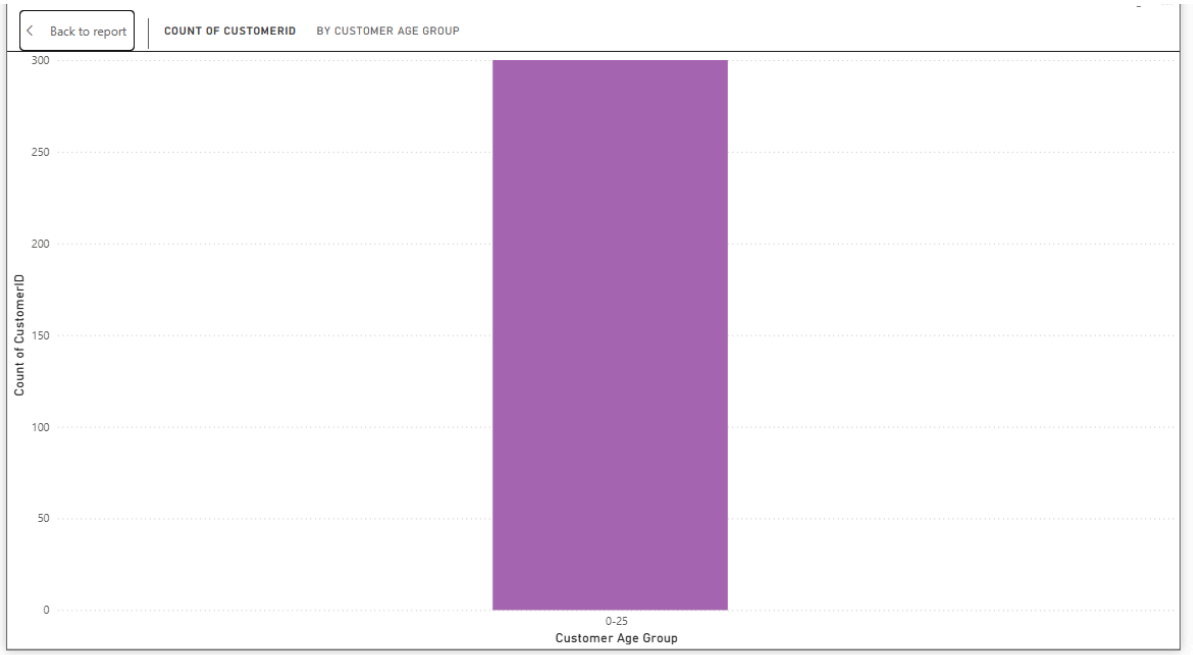
This KPI categorizes customers into different age brackets based on their date of birth.

The customer age calculation was performed using DAX functions that compare the customer's birth year with the current date.

```
Customer Age Group = SWITCH(TRUE(), [Customer Age]<=25, "0-25", [Customer Age]<=35, "26-35", [Customer Age]<=50,"36-50","51+")
```

A column chart was used because it provides a better view of customer concentration across age groups.

This visualization helps identify which age segments are most represented in the dataset and supports demographic analysis.



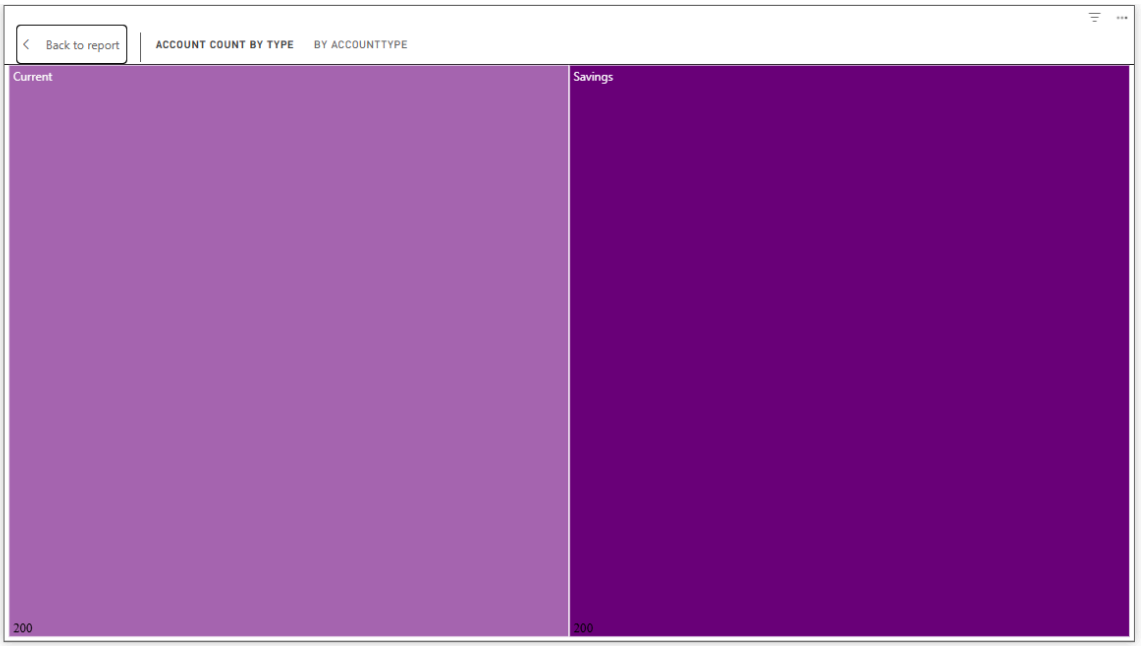
Accounts by Account Type

This KPI measures the number of accounts across different account categories such as Savings and Current accounts.

A treemap was used to compare account distribution visually.

This metric helps analyze:

- customer account preferences
- account concentration
- category level account trends



Total Balance by Name

This KPI compares total account balances across different customers to identify high value accounts, low balance accounts, and unusual balance patterns.

A bar chart was used to make customer level balance comparisons more clear and visually understandable.

This analysis helps highlight:

- account balance distribution
- customer financial behavior
- balance irregularities and outliers

Transaction Volume Trend

This KPI tracks the number of transactions occurring over time on a monthly basis.

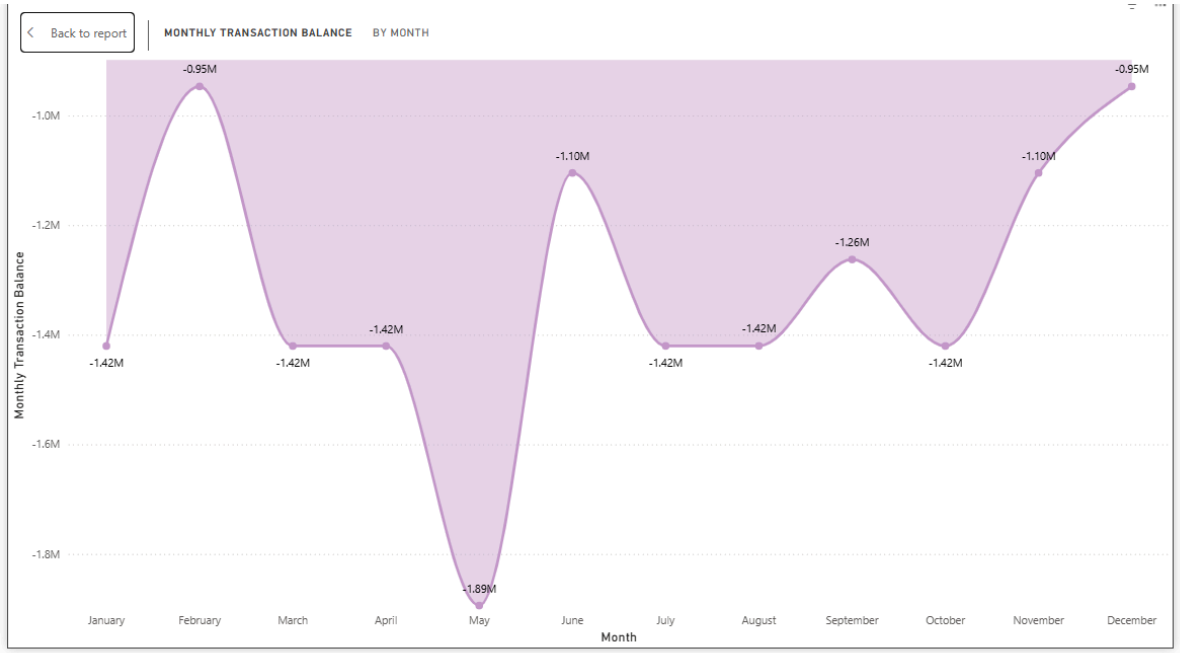
A line chart was selected because it is effective for visualizing time series trends and identifying transaction activity patterns.

The DAX calculation was designed to aggregate transaction counts month-wise while maintaining the overall time context.

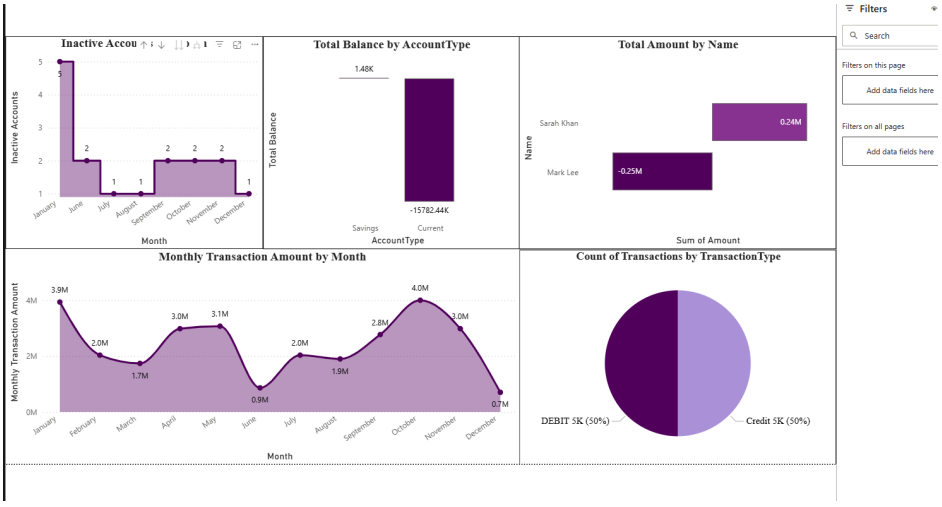
```
Monthly Transaction Balance = CALCULATE(SUM(CombinedBanking
Dataset[Balance]), ALLEXCEPT(CombinedBankingDataset, Combin
edBankingDataset[TransactionDate].[Month]))
```

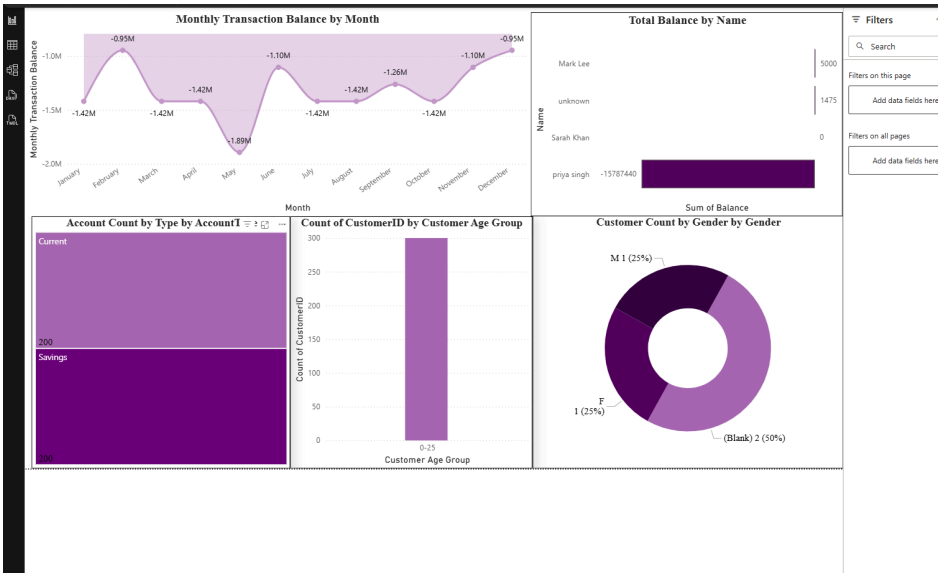
This analysis helps identify:

- transaction spikes
- seasonal trends
- activity fluctuations



Overall Dashboard





Final Insights

Transaction Activity Trends

Monthly transaction analysis showed fluctuations in transaction amounts across different months, with noticeable peaks during certain periods and lower activity toward the end of the year.

Equal Credit and Debit Distribution

The transaction type analysis showed an almost equal distribution between Credit and Debit transactions, indicating balanced transaction behavior within the dataset.

Customer Demographic Analysis

The dashboard highlighted customer distribution across gender and age groups, helping identify the dominant customer segments present in the banking data.

Account Type Distribution

Savings and Current accounts showed a nearly equal distribution in terms of account count, indicating balanced customer preference between account types.

Identification of Inactive Accounts

The inactive account KPI helped identify accounts with low or no recent transaction activity, which could indicate dormant users or reduced customer engagement.

Financial Irregularities & Outliers

Balance analysis revealed unusual financial patterns, including accounts with extremely large negative balances and zero balance accounts.

These outliers demonstrated how dashboards can help detect abnormal financial behavior, potential overdrafts, data inconsistencies, unusual account activity.